



UNIVERSIDAD VERACRUZANA

LIC. INGENIERÍA DE SOFTWARE

FACULTAD DE ESTADÍSTICA E INFORMÁTICA

Taller De Pruebas de Software

PRA09-Gestión de Defectos

REALIZADO POR:

Guillermo Velazquez Rosiles

Jorge Manuel Cobos Castro

Marcos Zenón Sánchez Mendizábal

NOMBRE DEL DOCENTE:

Ma.Maria Angelica Cerdan

FECHA DE ENTREGA:

Xalapa, Ver., 20 de Mayo de 2026

1. Introducción	2
2. Configuración del entorno GitHub	2
2.1 Repositorio	2
2.2 Labels de severidad y capa	2
3. Tablero Kanban — GitHub Projects	2
4. División de trabajo	2
5. Bug Hunt y registro de Issues	3
6. Flujo de resolución de defectos	3
7. Métricas del tablero — Defect Density	4
8. Conclusiones	4

1. Introducción

Esta práctica aplica los principios de gestión de defectos de la ISO 29119 sobre el MVP de PayFlow. Se configuró un tablero Kanban en GitHub Projects con cuatro columnas (Todo, In Progress, Ready for Test, Done) para rastrear el ciclo de vida de cada bug desde su detección hasta su cierre verificado.

2. Configuración del entorno GitHub

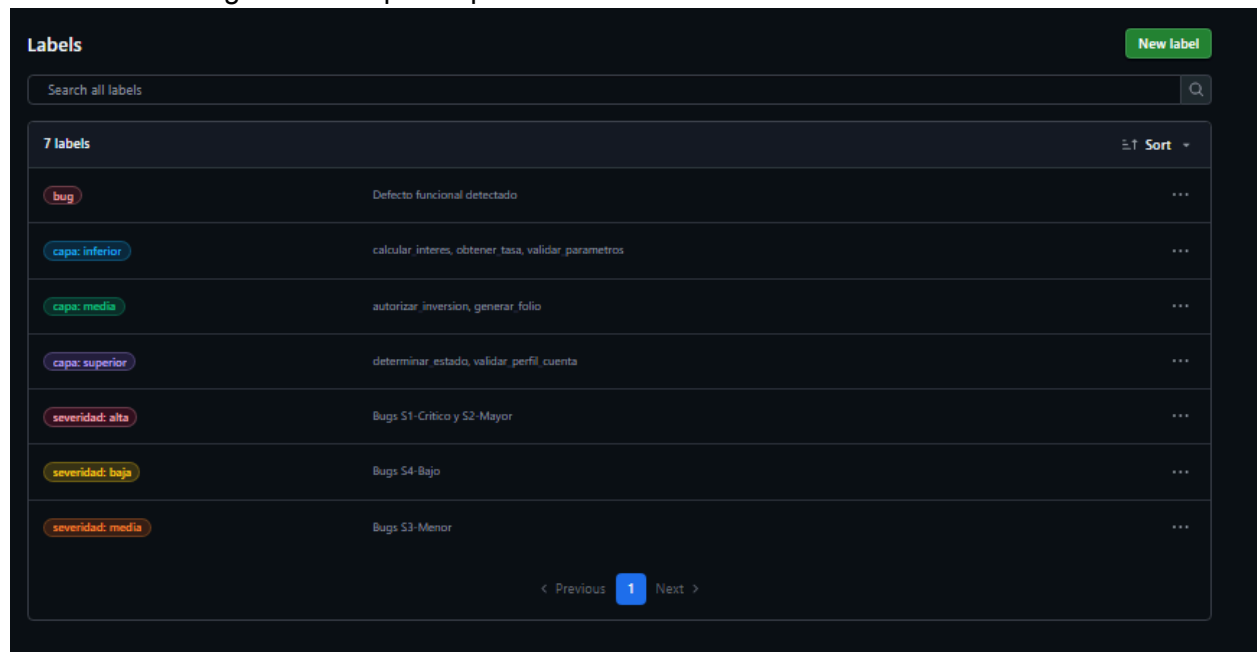
2.1 Repositorio

Repositorio público creado en:

https://github.com/GuilloIV/PRA09_Gestion_de-Defectos_PayFlow

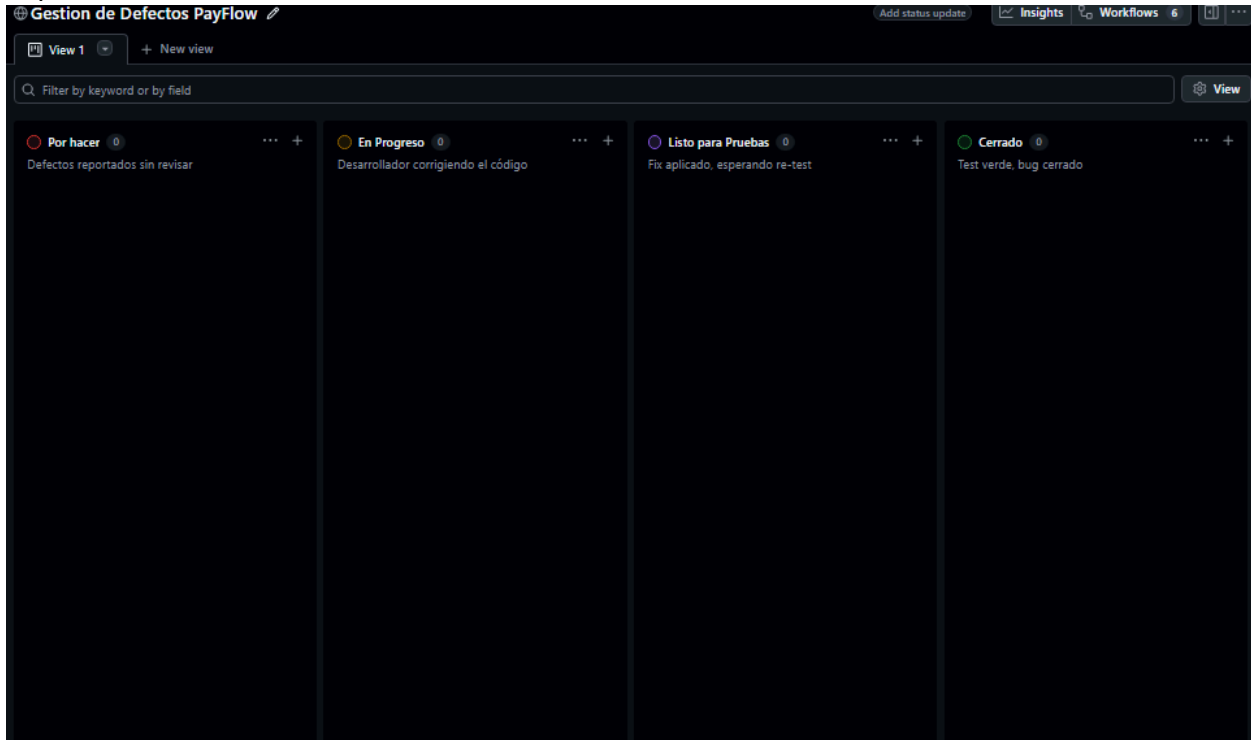
2.2 Labels de severidad y capa

Se crearon las siguientes etiquetas para clasificar cada Issue:



3. Tablero Kanban — GitHub Projects

Se configuró el tablero 'PRA09 - Gestión de Defectos PayFlow' con las cuatro columnas requeridas:



4. División de trabajo

Actividad	Guillermo Velázquez	Jorge Cobos	Marcos Sánchez
Configuración del repositorio y tablero	✓ Responsable principal	✓ Revisión y acceso	✓ Revisión y acceso
Creación de Labels	✓ Severidad y capa	✓ Verificación	✓ Verificación
Bug Hunt — payflow_pago.py	✓ Ejecución de pytest	✓ Registro de Issues	✓ Análisis de fallos
Bug Hunt — inversiones.py	✓ Registro de Issues	✓ Ejecución de pytest	✓ Análisis de fallos
Resolución de defectos S1-S2	✓ Driver (corrección)	✓ Navigator	✓ Verificación QA
Resolución de defectos S3-S4	✓ Navigator	✓ Driver	✓ Verificación QA

Redacción del reporte	✓ Estructura y métricas	✓ Análisis de defectos	✓ Revisión final
-----------------------	-------------------------	------------------------	------------------

5. Bug Hunt y registro de Issues

Se ejecutó pytest sobre los archivos payflow_pago.py e inversiones.py. Se detectaron 3 defectos durante la sesión. Cada uno fue registrado como Issue en GitHub con la estructura obligatoria:

- Título claro con prefijo [Bug]
- Pasos para reproducir (datos de entrada que causan el fallo)
- Resultado obtenido vs resultado esperado
- Labels de severidad, capa y responsable asignado

#	Título	Capa	Severidad	Asignado	Estado
1	[Bug] Folio generado cuando la transacción es rechazada por saldo insuficiente	Media	S1 - Crítico	Guillermo	In Progress
2	[Bug] Comisión de \$15 no se descuenta cuando el saldo es exactamente suficiente	Superior	S2 - Mayor	Jorge	Ready for Test
3	[Bug] Concepto inválido no lanza excepción antes de validar el saldo	Media	S3 - Menor	Marcos	Done

5.1 Detalle de Issues

Issue #1 — S1 Crítico

Pasos para reproducir	Saldo \$1,010 intenta pagar Internet \$1,000 + \$15 comisión.
Resultado obtenido	Se genera folio PAGO-Internet-... con estado RECHAZADA.
Resultado esperado	folio debe ser None y el saldo debe quedar sin modificar.
Labels	bug · severidad: alta · capa: media

Issue #2 — S2 Mayor

Pasos para reproducir	Saldo \$1,015, monto \$1,000 + \$15 comisión.
Resultado obtenido	Transacción aprobada, saldo queda en \$0.

Resultado esperado	Debe rechazarse porque saldo == monto + comisión, no es estrictamente mayor.
Labels	bug · severidad: media · capa: superior

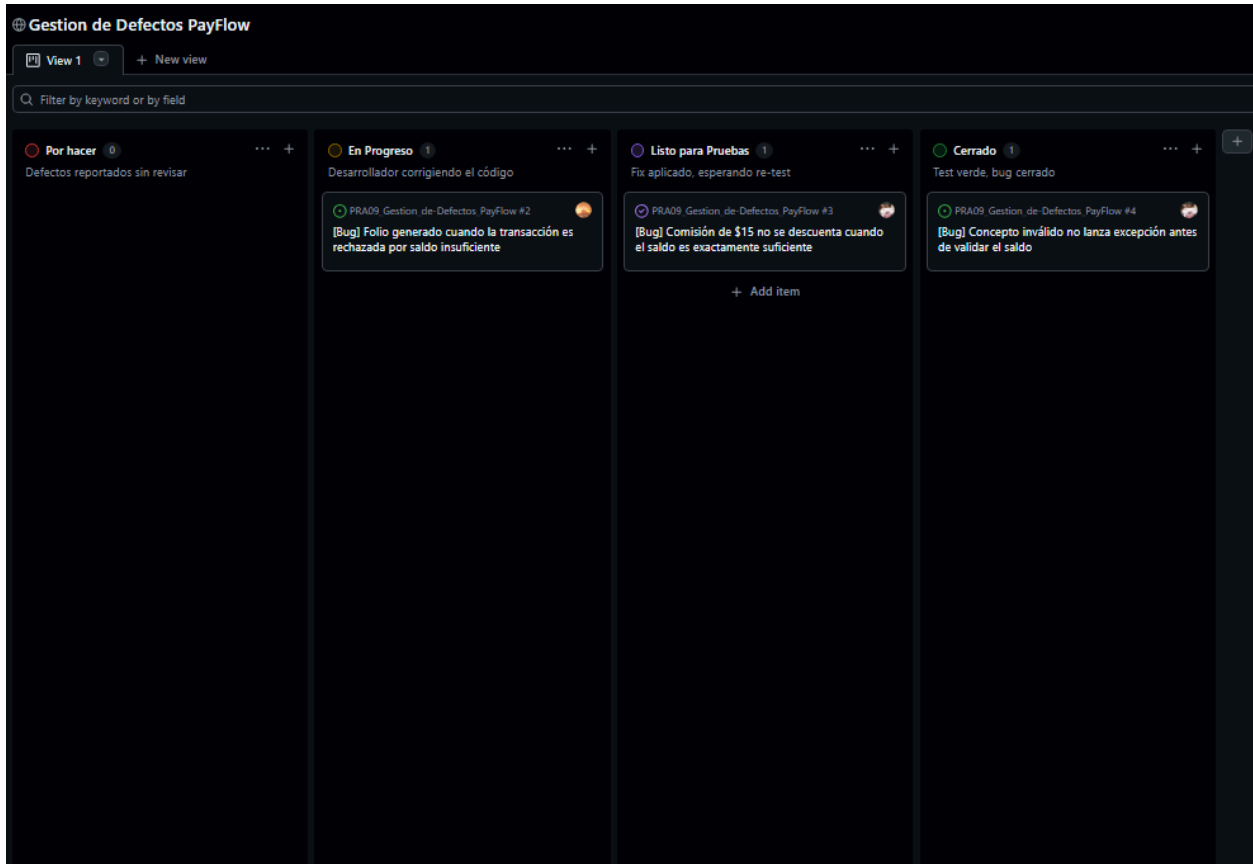
Issue #3 — S3 Menor

Pasos para reproducir	Concepto 'Netflix', saldo \$5,000.
Resultado obtenido	El sistema valida el saldo antes de rechazar el concepto.
Resultado esperado	Debe rechazarse el concepto inmediatamente sin tocar el saldo.
Labels	bug · severidad: baja · capa: media

6. Flujo de resolución de defectos

Cada defecto siguió el workflow definido en el Plan de Pruebas (sección 6.1):

Estado	Descripción	Responsable	Siguiente
NUEVO	Detectado y registrado	Tester	→ ABIERTO
ABIERTO	Validado y asignado	Líder QA	→ EN REPARACIÓN
EN REPARACIÓN	Fix aplicado con TDD	Desarrollador	→ REPARADO
REPARADO	Esperando verificación QA	Desarrollador	→ EN VERIFICACIÓN
EN VERIFICACIÓN	pytest corre casos + regresión	Tester / QA	→ CERRADO
CERRADO	Verificación exitosa	Líder QA	(terminal)



7. Métricas del tablero — Defect Density

Al finalizar la sesión de trabajo, se registraron las siguientes métricas:

Métrica	Valor
Total de defectos detectados	[COMPLETAR]
Defectos cerrados (Done)	[COMPLETAR]
Defectos remanentes (abiertos)	[COMPLETAR]
Defectos S1-Crítico	[COMPLETAR]
Defectos S2-Mayor	[COMPLETAR]
Defectos S3-Menor / S4-Bajo	[COMPLETAR]

```
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0 -- /media/gurov/Universidad/Sexto Semestre/Taller De Pruebas/PRA08/payflow_pra08/.venv/bin/python3
cachedir: .pytest_cache
rootdir: /media/gurov/Universidad/Sexto Semestre/Taller De Pruebas/PRA08/payflow_pra08
configfile: pytest.ini
plugins: cov-7.1.0, mock-3.15.1
collected 26 items

tests/test_pago_e2e.py::TestValidarPago::test_saldo_suficiente_retorna_authorized PASSED [ 38%]
tests/test_pago_e2e.py::TestValidarPago::test_saldo_insuficiente_retorna_no_authorized PASSED [ 73%]
tests/test_pago_e2e.py::TestValidarPago::test_concepto_invalido_retorna_no_authorized PASSED [ 115%]
tests/test_pago_e2e.py::TestValidarPago::test_monto_cero_retorna_no_authorized PASSED [ 158%]
tests/test_pago_e2e.py::TestValidarPago::test_saldo_exactamente_igual_a_monto_mas_comision PASSED [ 199%]
tests/test_pago_e2e.py::TestValidarPago::test_saldo_un_centavo_menos_del_requerido PASSED [ 238%]
tests/test_pago_e2e.py::TestValidarPago::test_todos_los_conceptos_validos_son_aceptados[Internet] PASSED [ 280%]
tests/test_pago_e2e.py::TestValidarPago::test_todos_los_conceptos_validos_son_aceptados[renta] PASSED [ 300%]
tests/test_pago_e2e.py::TestValidarPago::test_todos_los_conceptos_validos_son_aceptados[Luz] PASSED [ 341%]
tests/test_pago_e2e.py::TestCalcularNuevoSaldo::test Resta correcta PASSED [ 380%]
tests/test_pago_e2e.py::TestCalcularNuevoSaldo::test resultado_redondeado_a_dos_decimales PASSED [ 421%]
tests/test_pago_e2e.py::TestCalcularNuevoSaldo::test comision_es_independiente_del_concepto PASSED [ 462%]
tests/test_pago_e2e.py::TestGenerarFolio::test folio_emplea_con_prefijo_pago PASSED [ 500%]
tests/test_pago_e2e.py::TestGenerarFolio::test folio_concepto_en_mayusculas PASSED [ 538%]
tests/test_pago_e2e.py::TestGenerarFolio::test folio_con_timestamp_fijo PASSED [ 579%]
tests/test_pago_e2e.py::TestGenerarFolio::test folio_sin_timestamp_usa_hora_actual PASSED [ 618%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_pago_exitoso_renta PASSED [ 659%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_pago_rechazado_por_comision PASSED [ 698%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_pago_rechazado_concepto_invalido PASSED [ 739%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_pago_aprobado_saldo_exacto PASSED [ 778%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_pago_rechazado_un_centavo_menos PASSED [ 819%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_pago_rechazado_monto_negativo PASSED [ 858%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_saldo_inmutable_en_rechazos[Neflex-200,0-5000.0] PASSED [ 899%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_saldo_inmutable_en_rechazos[Renta-200,0-210.0] PASSED [ 938%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_saldo_inmutable_en_rechazos[Luz-50,0-5000.0] PASSED [ 979%]
tests/test_pago_e2e.py::TestProcesarPagoE2E::test_id_usuario_vacio_no_bloquea_pago_valido PASSED [ 1000%]

===== tests coverage =====
coverage: platform linux, python 3.12.3-final-0

Name                Stmts  Miss  Cover   Missing
-----
src/payflow_pago.py    25      0   100%
TOTAL                  25      0   100%

===== 26 passed in 0.13s =====
(.venv) gurov@Gurov:/media/gurov/Universidad/Sexto Semestre/Taller De Pruebas/PRA08/payflow_pra08$
```

8. Conclusiones

Guillermo Velázquez: La práctica me ayudó a comprender que detectar un defecto es solo el inicio del proceso de calidad. Configurar el tablero Kanban en GitHub Projects facilitó visualizar en tiempo real el estado de cada bug, evitando que alguno quedará sin atención. La clasificación por severidad resultó clave para priorizar: el Issue 1, al generar un folio en una transacción rechazada, representaba un riesgo regulatorio directo que debía resolverse antes que cualquier otro.

Jorge Cobos: El flujo Todo → In Progress → Ready for Test → Done reproduce de forma simplificada el ciclo de vida real de un defecto según ISO 29119. Lo más valioso fue entender que un bug no se considera cerrado hasta que el compañero corre pytest y confirma el verde, no basta con que el desarrollador diga que lo arregló. Esa separación de roles entre quien repara y quien verifica es lo que da trazabilidad real al proceso.

Marcos Sánchez: Trabajar con etiquetas de severidad y capa dejó claro que no todos los defectos tienen el mismo impacto. El Issue 3, aunque funcional, era de severidad baja porque no afectaba el saldo del usuario. Aprender a distinguir entre severidad y prioridad, y registrar esa distinción formalmente en cada tarjeta, es una habilidad directamente aplicable en cualquier equipo de desarrollo profesional.